CSC - 540 Project -1 Report

Team ID: 22

Team Members:

1. Rahul Shukla - rshukla3
2. Rishabh Agarwal - ragarwa9
3. Devashish Vachhani - dvachha
4. Kalgee Kotak - kkotak

Database Design

Introduction

The goal of the project was to design a relational database application for supporting a chain of car repair and service centers by a company. We achieved this by following an iterative process over ER diagram design, schema design, java implementation, defining triggers and constraints, and rigorous testing. In this report, we discuss our final database structure (tables, their relevance, their normal form and the respective functional dependencies), the constrained we identified within the project description, how we captured those constraints in our ER model (through checks and triggers, and through java implementation), and explanation of extensions to preliminary report.

Relations, Functional Dependencies and Normal Form

1. **Relation -** SERVICECENTER(<u>SC_ID</u>, ADDRESS, TELEPHONE_NO, STATE, OPEN_SAT, MIN_WAGE, MAX_WAGE, HOURLY_WAGE)

   Description:- This table stores all the information about the service centers. This can only be edited by Admin and Manager.

   FDs:-
   SC_ID → ADDRESS, TELEPHONE_NO, STATE, OPEN_SAT, MIN_WAGE, MAX_WAGE, HOURLY_WAGE

   Normal Form: BCNF

2. **Relation -** USERS(<u>USER_ID, SC_ID</u>, USERNAME, PASSWORD, ROLE)

Description:- This relation contains the information about all users irrespective of their role which may be Customer, or Employee(Manager, Receptionist or Mechanic). This table helps user keep track of the unique usernames and passwords for each user.

FDs :-
USER_ID, SC_ID → USERNAME, PASSWORD, ROLE
USERNAME → PASSWORD, ROLE

Normal Form - BCNF

3. **Relation -** EMPLOYEE(<u>USER_ID, SC_ID</u>, NAME, ADDRESS, EMAIL, PHONE_NO, ROLE)

   Description:- This table keeps track of all the employees irrespective of their roles. We store their personal information here.

   FDs :-
   USER_ID, SC_ID → NAME, ADDRESS, EMAIL, PHONE_NO, ROLE
   EMAIL → NAME, ADDRESS, PHONE_NO, ROLE

   Normal Form - BCNF

4. **Relation -** MANAGER(<u>USER_ID, SC_ID</u>, ANNUAL_SALARY)

   Description:- This table is used to store compensation information about all the managers. We have put a Unique constraint on the service center id attribute which assures that each service center has only one manager.

   FDs :-
   USER_ID, SC_ID → ANNUAL_SALARY

   Normal Form - BCNF

5. **Relation -** RECEPTIONIST(<u>USER_ID, SC_ID</u>, ANNUAL_SALARY)

   Description:- This relation contains the salary information about all the receptionists. Again, a unique constraint is placed on the service center id to ensure each service center only has one receptionist.

   FDs :-
   USER_ID, SC_ID → ANNUAL_SALARY

   Normal Form - BCNF

6. **Relation -** MECHANIC(USER_ID,SC_ID, HOURS_WORKED_WEEK1, HOURS_WORKED_WEEK2, HOURS_WORKED_WEEK3, HOURS_WORKED_WEEK4)

   Description:- This table contains the number of hours worked by each mechanic for all 4 weeks.

   FDs :-
   USER_ID, SC_ID → HOURS_WORKED_WEEK1, HOURS_WORKED_WEEK2, HOURS_WORKED_WEEK3, HOURS_WORKED_WEEK4

   Normal Form - BCNF

7. **Relation -** CARSERVICE(S_ID, NAME, TYPE)

   Description:- This table contains the information about all the services offered by the service centers. It stores the name of the service and type of the service(maintenance, repair, both).

   FDs :-
   S_ID → NAME, TYPE
   NAME → TYPE

   Normal Form - BCNF

8. **Relation -** MAINTAINANCE(S_ID, BUNDLE)

   Description:- This table stores the maintenance services and which bundle they belong to.

   FDs :-
   S_ID → BUNDLE

   Normal Form - BCNF

9. **Relation -** REPAIR(S_ID, CATEGORY)

   Description:- This table stores the repair services and the category they belong to.

   FDs :-
   S_ID → CATEGORY

Normal Form - BCNF

10. **Relation -** OFFEREDPRICE(S_ID, SC_ID, BRAND, PRICE)

    Description:- Since the price of any service depends on the service, service center, and brand of the vehicle, this table keeps track of all the prices.

    FDs :-
    S_ID, SC_ID, BRAND → PRICE

    Normal Form - BCNF

11. **Relation -** OFFEREDTIME(S_ID, BRAND, HRS)

    Description:- This table keeps track of the time taken to perform a particular service on a vehicle of a particular brand.

    FDs :-
    S_ID, BRAND → HRS

    Normal Form - BCNF

12. **Relation -** CUSTOMER(USER_ID, SC_ID, F_NAME, L_NAME, ADDRESS, EMAIL, PHONE_NO, STANDING, STATUS)

    Description:- This table contains all the information about the customers. It stores their personal information, their standing(whether they have paid all their invoices or not) and their status(If they have any vehicle or not).

    FDs :-
    USER_ID, SC_ID →F_NAME, L_NAME, ADDRESS, EMAIL, PHONE_NO, STANDING, STATUS
    EMAIL → F_NAME, L_NAME, ADDRESS, PHONE_NO, STANDING, STATUS

    Normal Form - BCNF

13. **Relation -** VEHICLE(VIN, USER_ID, SC_ID, BRAND, MILEAGE, YEAR, LAST_SERVICE)

    Description:- This table stores the information about all vehicles associated with all the users and service centers. We store the brand of the vehicle, Mileage, year and the last scheduled maintenance service

FDs :-
VIN → USER_ID, SC_ID, BRAND, MILEAGE, YEAR, LAST_SERVICE

Normal Form - BCNF

14. **Relation -** SERVICEEVENT(SE_ID, VIN, MECH_ID, SC_ID, CUST_ID, AMOUNT_PAID, AMOUNT_CHARGED, SERVICE_DATE, STATUS)

    Description:- This table stores all the service events (Invoices) and the associated details. We store the VIN of vehicle the services are performed on, customer who owns the vehicle, the mechanic who performed the service, payment status of the invoice, total cost, and the duration of the service.

    FDs :-
    SE_ID → VIN, MECH_ID, SC_ID, CUST_ID, AMOUNT_PAID, AMOUNT_CHARGED, SERVICE_DATE, STATUS
    VIN → SC_ID, CUST_ID

    Normal Form - 3NF

15. **Relation -** SERVICEEVENTDETAILS(SE_ID, S_ID)

    Description:- This table stores the individual services performed for each service event.

    FDs :- None

    Normal Form - BCNF

16. **Relation -** SLOTS(SLOT_ID, WEEK, SLOT_DAY, SLOTS)

    Description:- This is a static table that defines unique slot ids for all possible slots in a month. Slot id range from 1 (week 1, day 1, slot 1) to 240 (Week 1, Day 6 (Saturday), Slot 5 (Only 5 hours on a Saturday)).

    FDs :-
    SLOT_ID → WEEK, SLOT_DAY, SLOTS

    Normal Form - BCNF

17. **Relation -** MECHANICSCHEDULE(MS_ID, MECH_ID, SC_ID, SLOT_ID, AVAILABLE)

    Description:- This table keeps track of the availability of mechanics for all the slots in a month. It stores if a mechanic is available, working on a car, has taken time off, or if their respective service center is closed (Saturday).

FDs :-
MS_ID → MECH_ID, SC_ID, SLOT_ID, AVAILABLE
MECH_ID, SC_ID, SLOT_ID → AVAILABLE, MS_ID

Normal Form - BCNF

18. **Relation -** MECHANICSCHEDULESWAP(REGISTER_ID, MS_ID1,MS_ID2, APPROVED)

    Description:- This table keeps track of all the schedule swap requests for mechanics and their status. We store the mechanic schedule ids for both the mechanic requesting and mechanic being requested, which can give us the respective mechanic ids, service center ids, and slot ids.

    FDs :-
    REGISTER_ID → MS_ID1,MS_ID2, APPROVED

    Normal Form - BCNF

Constraints: What and How

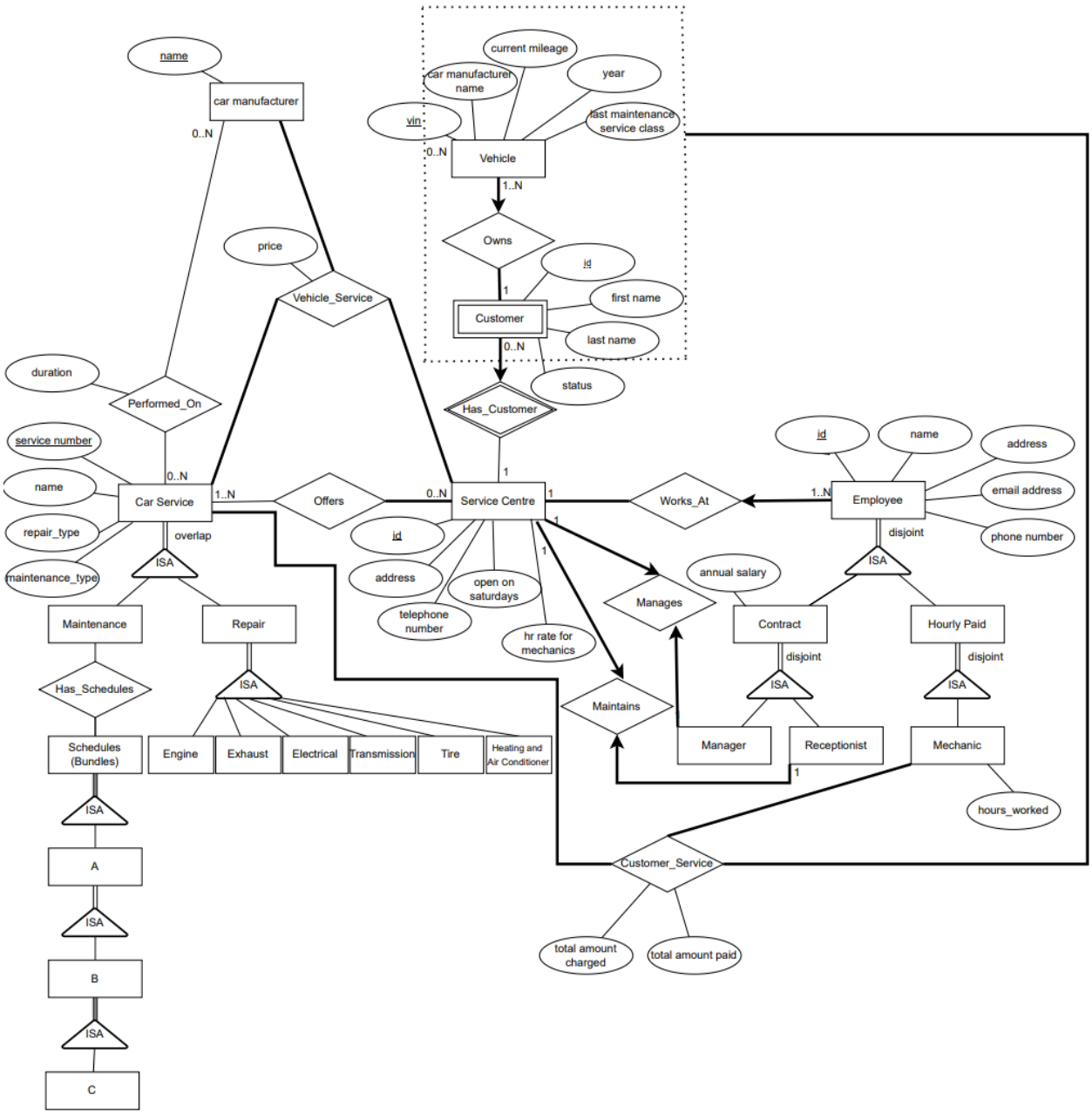
Sr. No	Constraint	ER/SQL/Java	How the constraint is captured
1	Some (not all) Service Centers are also open on Saturdays from 9am - 1pm	ER, SQL	Additional attribute open_sat in ServiceCenter Table captures this constraint
2	Each employee can only play one role at a time	ER, SQL	Assertion check constrained implemented through triggers
3	Individual services belong to only one of the 6 repair service subcategories	SQL	Not null constraint on category column in repair services table.
4	the duration for each service is the same in every store	ER, SQL	In the offered time table, the duration only depends on brand and service_id.
5	Each service also has a price and a time estimated for the service job, which is based on the car and the specific auto center	ER, SQL	In the offered price table, price depends on service center, service, and brand of the car (Depends on all three).

Sr. No	Constraint	ER/SQL/Java	How the constraint is captured
6	Maintenance - Scheduled Services are priced as a bundle	ER, SQL	The Bundles A, B, and C are considered separate services with their own price for each car and service center.
7	Maintenance services are usually done in a rotational manner i.e. after a Schedule A maintenance service, the next maintenance will be Schedule B, then Schedule C. After Schedule C, it restarts at Schedule A.	Java,SQL	We store the last maintenance service performed on a vehicle in the vehicle table. We then use Java logic to automatically determine which would be the next maintenance service be.
8	Each mechanic works no more than 50 hours a week	ER, SQL	Check constraint on the hours worked attribute in the mechanic table. It has 4 columns capturing the hours worked in each week and has constraint <= 50.
9	A mechanic may ask for and may be given time off, as long as there are always at least 3 mechanics present at any given time	SQL, Java	We find out the number of mechanics who are working during the requested time slots using a query, and use java to make sure the mechanic cannot request time off if the result of the query is less than 3.
10	User_id for Customer/Employee is locally unique for Store	ER, SQL	User_id and serviceCenter_id combined is implemented as foreign key to Employee, Mechanic, Receptionist, Customer and Manager table
11	Each service Center has only one manager	SQL, Java	In the Manager table we have a unique constraint on serviceCenterId.
12	Each service Center has only one Receptionist	SQL, Java	In the Receptionist table we have a unique constraint on serviceCenterId.
13	A Customer has id (an integer) that is unique with respect to a specific center.	ER, SQL	In customer table,(user_id, serviceCenter_id) is primary key
14	A new customer can be added only by the receptionist.	Java	User with role Receptionist has access to the page through which Customer is added to database

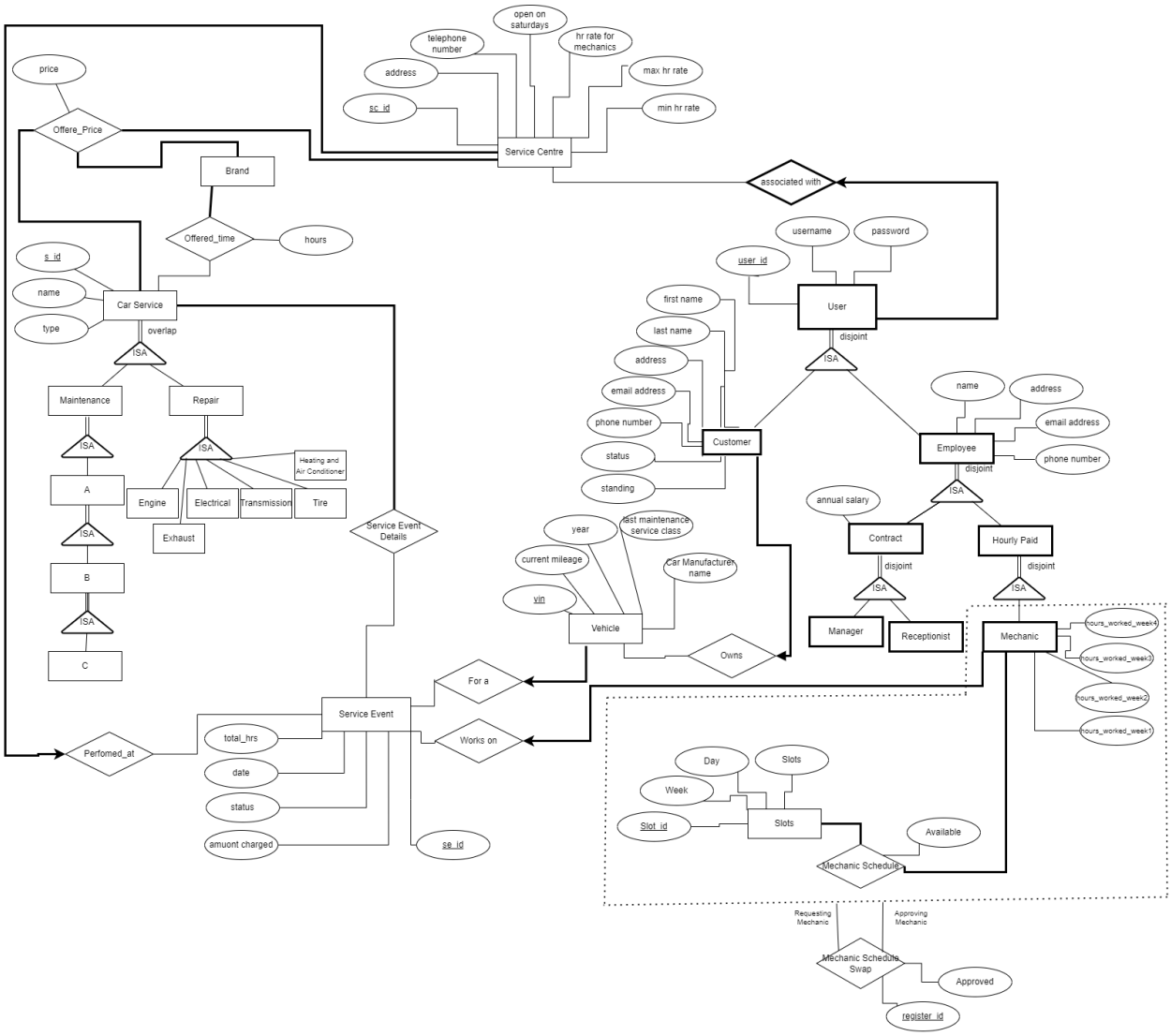
Sr. No	Constraint	ER/SQL/Java	How the constraint is captured
15	Every customer is associated with one specific service store	SQL , ER	Foreign Key constraint on the service center id attribute in Customer Table.
16	Customers become “inactive” if their profiles no longer include any vehicles	SQL, Java	Assertion check constrained implemented through triggers.
17	Each customer has a status field stating if their account is in good standing or not (i.e. has an outstanding balance).	SQL, Java	Assertion check constrained implemented through triggers.
18	Scheduling will need to avoid double-booking or overbooking a mechanic i.e. > 50hrs or booking them while they are on vacation	SQL	ScheduleEvent has a mechanic_id attribute that captures only one mechanic information and updates MechanicSchedule for the mechanic_id. Mechanics cannot be booked if the working attribute is false in MechanicSchedule (i.e. on vacation).
19	Also, scheduling should not book on a Saturday if the store is not open on that day.	SQL, Java	For ServiceCenter that are closed on Saturday, MechanicSchedule working attribute is marked null. We have a check constraint that ensures available attributes cannot be marked true until unless working is true.
20	If they want a maintenance service done, the system automatically selects the correct service schedule (‘A’, ‘B’, ‘C’) based on the car’s maintenance history which is on record	SQL, Java	We store the last maintenance service record in the Vehicle table. We then use Java logic to automatically determine which would be the next maintenance service.
21	Each service event (all services scheduled by a customer in a single visit to the interface) is handled by a single mechanic.	ER, SQL	When a new entry is inserted into ScheduleEvent only 1 mechanic id associated with it.

Changes to the ER diagram from Milestone 1 to Final

Milestone 1:



Final:



1. Entity
    a. User is introduced to keep track of all users (Employees and Customers)
    b. Car_Manufacturer is changed to Brand
    c. Customer_Service is changed to Service_Event
    d. Slots is introduced to keep track of slots

2. Relationships
    a. Vehicle_Service is changed to Offered_Price
    b. Performed_On is changed to Offered_Time
    c. Service_Event_Details is introduced to keep track of services in an invoice
    d. Mechanic_Schedule is introduced to track mechanic availability

e. Mechanic_Schedule_Swap is introduced to track swap requests